

Taller Express y otras cosas



Nombre: Genaro Mercado Morones

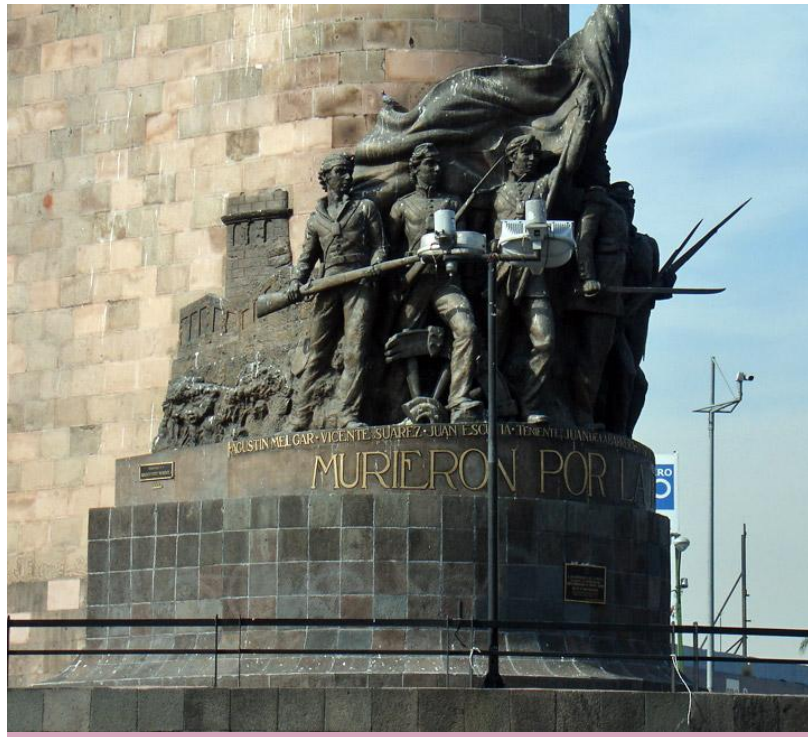
Matrícula: A01647350

Película favorita y razón: Lego Batman porque crea una narrativa muy apegada a lo que es Batman pero en un ambiente creativo y divertido.

El objetivo de la actividad es explorar y conocer más sobre el desarrollo web del lado del backend.

Al final de la actividad deberás hacer un PDF con este documento. El PDF deberá tener un enlace en [miarchivo.html](#)

- **Ejercicio -1: Una pregunta**



¿Conoces ese lugar? ¿Cuál es su nombre oficial? ¿Con qué otro nombre se conoce?

No conozco el lugar, pero se llama la glorieta de los niños héroes, supuestamente conocido también como la glorieta de los y las desaparecidos.

- **Ejercicio 0: El pasado**

En las semanas previas vimos la forma de consumir un API desde el cliente (el navegador). Ahora consumirás el **API desde el servidor**. Usa javascript y nodejs.

¹ This image was originally posted to Flickr by kapela_07 at <https://www.flickr.com/photos/48569350@N05/4451995199>. It was reviewed on 24 March 2010 by FlickreviewR and was confirmed to be licensed under the terms of the cc-by-2.0.

Selecciona un API que no requiera autenticación.

El servidor deberá consumir el API, procesarlo de alguna manera (ejemplo: seleccionar un dato) y posteriormente entregar ese dato (o datos) al cliente (navegador).

Sube tu archivo a tu repositorio.

Escribe aquí abajo el enlace al archivo en tu repositorio:

<https://genarinh0.github.io/ActividadExpress/EjerciciosExpress.html>

¿Cuál fue el principal problema para resolver este ejercicio? ¿Cuál fue el problema que tuvo una de las personas en tu equipo? Explica cómo resolvió esa persona ese problema.

El principal problema que tuve creo que fue encontrar un API que no requiriera la autenticación y fuera algo simple para implementar. El problema que tuvo un compañero fue el como acceder a los datos que quería, lo que hizo fue hacer el request y en la consola ver que devolvía la API. De esa manera, pudo ver la estructura que se entregaba y así acceder solo a los datos que quería.

- **Ejercicio 1: Simplicidad**

¿ Recuerdas cómo **instalar un módulo** en node.js? Escribe la instrucción que se usa:
`npm install nombre_del_modulo`

- **Ejercicio 2: Express es**

Investiga qué es express (en el contexto de desarrollo web en nodejs). Explica a un **niñ@** qué es express, cómo se usa, para qué sirve. Escribe ese texto aquí. Incluye también **las referencias** de tu investigación.

Express es como una caja de herramientas que puedes usar para crear un lugar a donde las personas pueden ir para que les hables o te hablen a ti. Piensa esta caja de herramientas como las herramientas para construir una casa. A esa casa (servidor) llegan tus amigos y te pueden pedir o dar cosas que tu puedes guardar en un lugar escondido en tu casa (base de datos).

Introducción a Express/Node - Aprende desarrollo web | MDN. (2024, December 21). Mozilla.org.

https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction

W3Schools.com. (2026). W3schools.com.

https://www.w3schools.com/nodejs/nodejs_express.asp

Dile a una persona de la clase (que no esté en tu mesa) que te explique qué es express y qué ventajas te podría dar su uso. Graba el audio de la explicación.

Audio en pagina donde hice lo de la API:

<https://genarinh0.github.io/ActividadExpress/EjerciciosExpress.html>

¿Crees que realizó una buena explicación? ¿Por qué?

Si me gusto la analogía que usan para explicar pues de esta manera se vuelve un concepto más fácil de asimilar con menos cosas técnicas y solo lo que es en teoría express.

- **Ejercicio 3: Desde un libro**

Investiga una definición en un libro (digital o físico) **de una API**. Incluye la referencia del libro.

An API is the interface that a software program presents to other programs, to humans, and, in the case of web APIs, to the world via the internet.

Jin, B., Sahni, S., & Shevat, A. (n.d.). *Designing Web APIs BUILDING APIS THAT DEVELOPERS LOVE*. Retrieved March 25, 2026, from

<https://ebooks.karbust.me/Technology/Designing%20Web%20APIs%20-%20Brenda%20Jin,%20Saurabh%20Sahni,%20Amir%20Shevat-OReilly-9781492026921.pdf>

Escribe un ejemplo de un endpoint de tu proyecto.

GET /api/publicaciones

- **Ejercicio 4: Instala express**

Escribe la instrucción que se usa para **instalar el módulo de express**:

npm install express

- **Ejercicio 5: Uso de express**

Ejecuta y explica qué hace el siguiente código:

```
import express from 'express';
```

```
const app = express();
```

```
app.listen(1984, () => {  
  console.log('Up and up');  
});
```

Este código básicamente hace lo que hicimos en la actividad del servidor, pero mucho más simplificado usando el framework de express. La primera línea importa la librería que instalamos con node, después creamos una variable que crea una instancia de express que es como un objeto. Enseguida llamamos al método listen de la variable anterior que escucha las peticiones en el puerto ingresado. Toma dos argumentos, el puerto y la función que ejecutara al iniciarse correctamente el servidor.

- **Ejercicio 6: Uso de express++**

Ejecuta y explica qué hace el siguiente código:

```
app.get('/bienvenida', (req, res) => {  
  res.send('Esto no es una página html');  
});  
  
app.get('/otraBienvenida', (req, res) => {  
  res.sendFile('bienvenida.html');  
});
```

El código crea dos rutas con método http get usando el objeto app declarado anteriormente. El método toma dos argumentos, el primero siendo la URL a la que se debe de visitar y la segunda una función que se ejecuta cuando se entra a la URL. Con res.send() express detecta que mandas y lo muestra en el navegador. En el primero mandamos texto simple y en el segundo una página usando res.sendFile(), aunque aparece un error por la ruta de esta.

En caso de que alguno de estos te marque error, soluciona el problema y explica la forma de solucionarlo:

El problema esta en la forma que se accede al archivo, resulta que se necesita una ruta absoluta para acceder correctamente al archivo. Para ello uso dos librerías: path y url, por el motivo de que estamos usando un tipo de modulo que como que tiene diferencias con JavaScript normal. Con ellas, accedemos a la ruta necesaria y usamos el método de path llamado path.join() para unir la ruta y hacer nuestro endpoint.

- **Ejercicio 7: Recuerdo de imagen**

Es posible que la imagen no se podía ver en <https://github.com/sqiomatec/act2026>. Un servidor entrega respuestas a solicitudes realizadas por un cliente. Cuando el cliente solicita una imagen (u otro tipo de archivo) el servidor tiene que saber responder.

Express nos propone crear un directorio para los archivos que queremos usar, por ejemplo imágenes o archivos de hojas de estilo.

Investiga qué es `express.static`.

Escribe tu investigación

Esta es una función que proviene de express la cual nos ayuda a dar archivos estáticos como lo son las imágenes, o archivos con código como en html, css o js al cliente. El problema que hizo necesario esta función es que usualmente requieres una ruta específica para cada archivo estático que quieres mandar. Ya con la función no necesitas hacer esto, es un proceso mucho mas simple solo creas un folder donde le dices a express que allí estarán tus archivos estáticos a mandar y listo.

Referencia:

- GeeksforGeeks. (2020, July 3). *Express `express.static()` Function*. GeeksforGeeks.

<https://www.geeksforgeeks.org/web-tech/express-js-express-static-function/>

- **Ejercicio 8: Película**

Pregunta a una persona de otro equipo su película favorita y sus razones.

¿Ya viste esa película? ¿Te gusta? ¿La verías si no la has visto? ¿Por qué?

La respuesta de mi compañero fue la película de interestelar, ya la he visto varias veces y la verdad si me gusta mucho. Es una gran película, pero creo que de tanto que la he visto ya siento que no me llama igual la atención como antes. Por ello mi película favorita es otra, aunque bien pudiera ser esta misma.

- **Ejercicio 9: Transformación**

Usa express para transformar todo el código de servidor.js, nombra el archivo ahora servidor_express.js

Haz comentarios de qué te parece *express* en el mismo código.

Haz que se vea una foto de un árbol de tu universidad con el nombre científico del árbol.

Escribe aquí abajo el enlace al archivo en tu repositorio:

https://genarinh0.github.io/Actividad5/servidor_express.js

- **Ejercicio 10: Verbos**

En el contexto de desarrollo web, explica los siguientes verbos GET, POST, PUT, DELETE

Estos verbos son los métodos HTTP mas comunes, donde cada uno indica por su mismo nombre que va a realizar y por ello nos evitamos confusiones a la hora de definir nuestros endpoints. GET es un verbo que se traduce a obtener, por ello el usarlo significa que esperas obtener o agarrar algo de la API solamente, sin alterar ni mandar nada. POST traduce a mandar así que indica que tu vas a mandarle algo al servidor para que algo se vaya a crear o se vaya a guardar, por ejemplo. PUT traduce a poner, pero creo que se encamina mas a actualizar. Con este método indicas que vas a hacer un cambio en un dato que ya existe. Por último, DELETE, que es borrar. Lo usamos cuando tenemos la intención de borrar algo completamente en el servidor que ya existe.

Explica los siguientes códigos, en el contexto de desarrollo web:

1xx: Son respuestas informativas del servidor, solamente son para indicarle al cliente el progreso de la solicitud, pero no genera o modifica nada. Por ejemplo, algunos significan cosas como procesando, cambiando protocolos, etc.

2xx: Indican cosas positivas o al menos cosas no completamente negativas, como que el servidor acepto, creo o que todo esta en orden.

3xx: Especifican una redirección, un recurso que se mueve a otra dirección. Indican cosas como que se movió el recurso, que se encontró en otra URL, entre otros.

4xx: Estos códigos son error como solicitudes no validas, URL inexistentes (como el famoso 404), credenciales incorrectas, etc. Considero que son lo contrario a los 2XX.

5xx: Indican que hubo problemas en el lado del servidor, algo sobre lo que el cliente no tiene control. Ejemplos son servicios inactivos, errores internos, almacenamiento insuficiente, y demás.

Referencia:

- Hosting, S. W. (2020, September 29). Códigos de estado HTTP explicados. SiteGround. <https://es.siteground.com/kb/codigos-error-http-explicados/>

Explica con un ejemplo (relacionado con tu proyecto/reto) cómo se atendería una solicitud de cada uno de esos verbos usando express. Incluye al menos una respuesta 404, 201 y 200.

GET: para recabar las publicaciones. Si se pueden obtener bien entonces mandamos un 200, sino un 500 porque fue error del servidor. Se vería así:

```
app.get('api/publicaciones', async (req, res) => {
  try {
    const publicaciones = await getPublicaciones();
    res.status(200).json(publicaciones);
  } catch (error) {
    console.log(error);
    res.status(500).json({ error: 'Error al obtener publicaciones' });
  }
});
```

POST: para registrar una cuenta nueva. Si se llena bien los datos entonces se crea el usuario y regresamos 201 indicando recurso creado exitosamente:

```
app.post('api/usuario', async (req, res) => {
  await crearUsuario(req.body);
  res.status(201).send('Usuario creado exitosamente');
});
```

PUT: para actualizar el estado de una publicación o cambiar los datos del mismo. Si se logra entonces mandamos un 200:

```
app.put('api/publicaciones/:id', async (req, res) => {
  const { id } = req.params;
  await actualizarPublicacion(id, req.body);
  res.status(200).send('Publicacion actualizada');
});
```

DELETE: cuando queramos quitar una publicación de nuestros favoritos, o guardados entonces usamos este método. Nuevamente usamos un 200 si todo bien, y si no se encuentra la publicación en tus guardados entonces lanzamos un 404 porque significa recurso no encontrado:

```
app.delete('api/guardados/:id', async (req, res) => {
  const { id } = req.params;
  const borrado = await borrarGuardado(id);

  if (!borrado) {
```



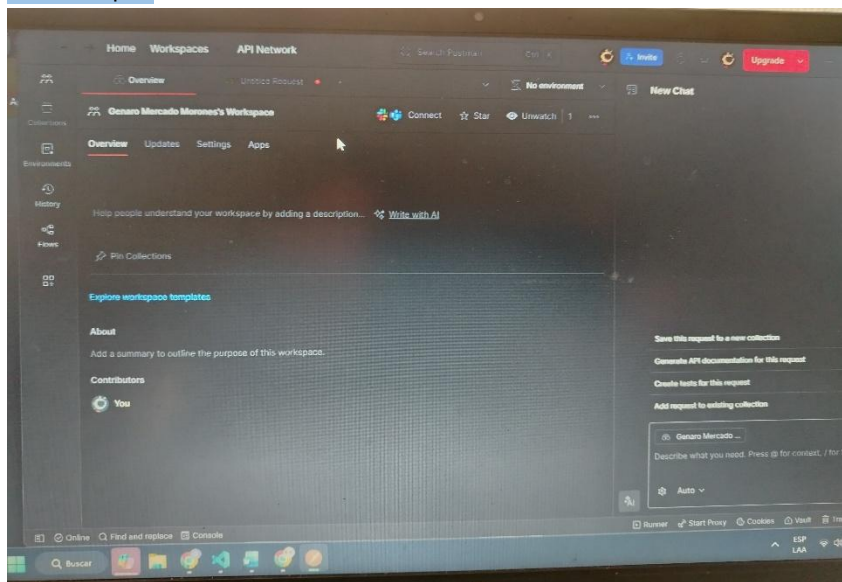
```
        return res.status(404).send('Guardado no encontrado');
    }

    res.status(200).send('Publicacion Borrada de Guardados');
});
```

- **Ejercicio 11: Postman**

En equipo con tu mesa, instalen postman en cada una de sus computadoras. Tomen una misma foto en la que aparezcan todas las pantallas con postman abierto.

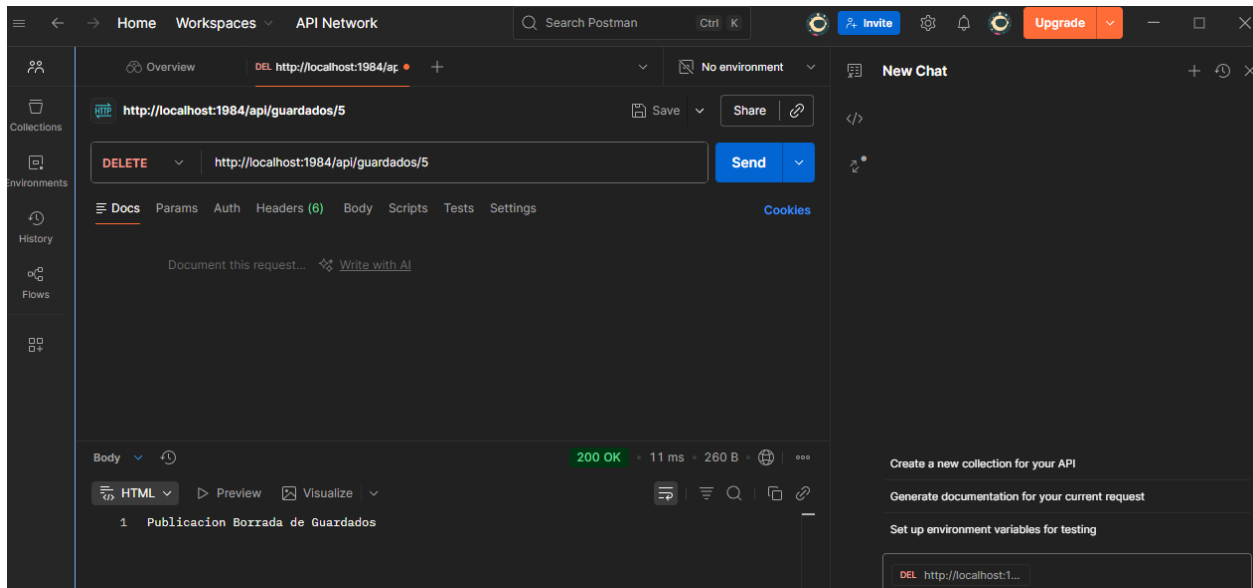
Foto aquí:



Explica para qué sirve Postman

Es una aplicación que nos permite simular un cliente que está haciendo peticiones al servidor que tenemos para poder probar que funcione adecuadamente. Con el puedes elegir que endpoint usar de tu API y si es necesario también mandar datos.

Prueba lo que hiciste en el ejercicio 10 con Postman. Pon una captura de pantalla usando el DELETE.



- **Ejercicio 12: Parámetros**

Inventa un ejercicio (que esté relacionado con mascotas/usuarios) para que uses parámetros en las rutas (por ejemplo /getMascotas)

Obtén todas las publicaciones hechas aplicando filtros como de ubicación, raza y especie, por medio de parámetros de ruta.

Soluciona ese ejercicio

Escribe aquí abajo el enlace al archivo en tu repositorio:

<https://genarinh0.github.io/ActividadExpress/parametros.js>

Compara ese ejercicio con el de otra persona de la clase. ¿Qué ejercicio ayuda mejor a comprender el uso de parámetros y por qué?

Creo que el mío estuvo un poco mejor porque te permite usar varios parámetros con el uso de &, ya que esto es bastante importante al usar parámetros de query.

Encontré que hay diferencia entre estos y los de path, y para la actividad que cree, que de hecho tendremos que implementar en nuestro sitio, lo mejor era usar estos de query así que me ayudo a comprenderlos mejor–.

- **Ejercicio 13: npx**

Investiga con alguien más lo siguiente: npx express-generator. Importante haz un directorio diferente si quieres ejecutar el comando.

Explica lo que genera y hace ese comando.

Este comando básicamente te evita gastar tiempo haciéndote un esqueleto de tu aplicación express. Por eso se llama generador de express. Viene con configuración, middleware y directorios predeterminados para que desarrolles tu app web mucho más rápido sin hacer ese labor.

Explica de manera básica y conceptualmente qué archivos se generaron.

Se generaron 4 carpetas y dos archivos individuales. La carpeta bin contiene el código para iniciar el servidor en un archivo www; la carpeta public es donde puedes guardar tus archivos estáticos a los que el navegador puede acceder; routes es para definir las rutas de tus endpoints de tu aplicación; y la carpeta views es para las plantillas HTML renderizadas desde el servidor (lo investigue). El package.json sirve igual para poner dependencias y configuraciones, y por ultimo el app.js es donde se configura lo esencial de express como sus rutas, el middleware y la definición del mismo express.

- **Ejercicio 14: Explicar**

Explica a una niña o un niño de 10 años para qué sirven frameworks como Angular, React, Vue, o Svelte.

Son como unas cajas de herramientas, pero donde cada una tiene herramientas diferentes y tu escoges cual caja vas a usar según lo que ocupes. Tienen un parecido de que todas estas herramientas ayudan para la misma cosa, para algo así como dibujar en una pantalla de una computadora o celular. Unas cajas son más grandes, otras más fáciles de usar, y en general cada una aporta cosas distintas así que usas la que más te convenga.